

# Deployment Runbook

---

- [NetworkPeer Cloud Infrastructure & Database Setup Runbook](#)
  - [Table of Contents](#)
  - [1. Prerequisites & Access](#)
    - [1.1 Required Tools](#)
    - [1.2 AWS Credentials \(Organization Provided\)](#)
    - [1.3 Required IAM Permissions](#)
  - [2. AWS Account Setup](#)
    - [2.1 Region Selection](#)
    - [2.2 Service Quotas \(Request Increases if Needed\)](#)
    - [2.3 SNS SMS Production Access \(CRITICAL\)](#)
  - [3. Terraform Backend Configuration](#)
    - [3.1 Create Backend Resources \(One-time\)](#)
    - [3.2 Configure Main Backend](#)
    - [3.3 Initialize](#)
  - [4. Infrastructure Deployment](#)
    - [4.1 Prepare Variables](#)
    - [4.2 Key Variables \(terraform.tfvars\)](#)
    - [4.3 Plan & Apply](#)
    - [4.4 Expected Outputs](#)
  - [5. Database Provisioning & Migrations](#)
    - [5.1 Verify RDS Connectivity](#)
    - [5.2 Create Application Database & User](#)
    - [5.3 Store Credentials in Secrets Manager](#)
    - [5.4 Run Migrations](#)
    - [5.5 Critical Migration: Cognito Identity Mapping](#)
    - [5.6 Seed Data \(Optional - Demo\)](#)
  - [6. Cognito Custom Auth Configuration](#)
    - [6.1 Verify Terraform-Created Resources](#)
    - [6.2 Verify Lambda Triggers](#)
    - [6.3 Test Custom Auth Flow \(CLI\)](#)

- 6.4 Configure Groups (CLIENT, WORKER, ADMIN)
- 7. SNS SMS Setup
  - 7.1 Verify SNS Configuration
  - 7.2 Configure Origination Identity (Required for Production)
  - 7.3 Test SMS Delivery
- 8. Application Deployment
  - 8.1 Build Docker Images
  - 8.2 Update ECS Service
  - 8.3 Deploy Web Frontend
  - 8.4 Configure Environment Variables (ECS Task Definition)
- 9. Monitoring & Alerting
  - 9.1 Key Metrics to Monitor
  - 9.2 Create Dashboards
  - 9.3 Log Insights Queries
- 10. Rollback Procedures
  - 10.1 Application Rollback
  - 10.2 Database Rollback
  - 10.3 Terraform Rollback
- 11. Production Checklist (Pre-Go-Live)
  - Infrastructure
  - Cognito
  - Application
  - Security
  - Monitoring
  - Mobile Apps
- 12. Cost Estimation (Monthly, us-east-1)
- 13. Emergency Contacts

# NetworkPeer Cloud Infrastructure & Database Setup Runbook

---

**Version:** 1.3 (Cognito Custom Auth)

**Date:** 2026-09-02

**Target:** Zero-to-Production on AWS

**Prerequisites:** AWS CLI v2, Terraform  $\geq$  1.5, Docker, Node.js 20+, Organization AWS Credentials

---

## Table of Contents

---

1. [Prerequisites & Access](#)
  2. [AWS Account Setup](#)
  3. [Terraform Backend Configuration](#)
  4. [Infrastructure Deployment](#)
  5. [Database Provisioning & Migrations](#)
  6. [Cognito Custom Auth Configuration](#)
  7. [SNS SMS Setup](#)
  8. [Application Deployment](#)
  9. [Monitoring & Alerting](#)
  10. [Rollback Procedures](#)
-

# 1. Prerequisites & Access

---

## 1.1 Required Tools

```
# Verify versions
aws --version      # >= 2.15
terraform --version # >= 1.5
docker --version   # >= 24.0
node --version     # >= 20.0
npm --version      # >= 10.0
```

## 1.2 AWS Credentials (Organization Provided)

```
# Option 1: AWS SSO (Recommended)
aws configure sso --profile networkpeer-prod

# Option 2: IAM User with MFA
aws configure --profile networkpeer-prod
# Enter: Access Key ID, Secret Access Key, Region (us-east-1), Output (json)
```

## 1.3 Required IAM Permissions

The deployment role needs: - `AdministratorAccess` (for initial bootstrap) OR fine-grained: - EC2, ECS, RDS, ElastiCache, S3, Cognito, SNS, IAM, CloudWatch, Route53, ACM, Lambda, CloudFormation - `iam:CreateRole`, `iam:AttachRolePolicy`, `iam:PutRolePolicy` for service roles

---

# 2. AWS Account Setup

---

## 2.1 Region Selection

```
export AWS_REGION=us-east-1 # Primary region
export AWS_PROFILE=networkpeer-prod
```

## 2.2 Service Quotas (Request Increases if Needed)

| Service     | Quota                       | Required    | Current         |
|-------------|-----------------------------|-------------|-----------------|
| EC2         | Running On-Demand Instances | 20+         | Check           |
| RDS         | DB Instances                | 5+          | Check           |
| ElastiCache | Nodes                       | 5+          | Check           |
| Cognito     | User Pools                  | 10+         | Check           |
| SNS         | SMS Spend Limit             | \$10+/month | <b>Critical</b> |
| Lambda      | Concurrent Executions       | 100+        | Check           |

**Request increases via:** AWS Console → Service Quotas → Request quota increase

## 2.3 SNS SMS Production Access (CRITICAL)

```
# Check current status
aws sns get-sms-attributes --profile networkpeer-prod

# If in sandbox, request production access:
# AWS Console → SNS → Text messaging (SMS) → Request production access
# Provide: Use case, volume estimate, opt-in proof
```

## 3. Terraform Backend Configuration

---

### 3.1 Create Backend Resources (One-time)

```
cd /Users/adityasharma/Desktop/NETWORKPEER/infra/terraform

# Create S3 bucket + DynamoDB table for state locking
cat > backend-bootstrap/main.tf <<'EOF'
resource "aws_s3_bucket" "terraform_state" {
  bucket = "networkpeer-terraform-state-prod"
  versioning { enabled = true }
  server_side_encryption_configuration {
    rule { apply_server_side_encryption_by_default { sse_algorithm =
      "AES256" } }
  }
  lifecycle { prevent_destroy = true }
}

resource "aws_dynamodb_table" "terraform_locks" {
  name          = "networkpeer-terraform-locks"
  billing_mode  = "PAY_PER_REQUEST"
  hash_key      = "LockID"
  attribute { name = "LockID", type = "S" }
  ttl { attribute_name = "TTL", enabled = true }
}
EOF

terraform -chdir=backend-bootstrap init && terraform -chdir=backend-bootstrap
  apply
```

## 3.2 Configure Main Backend

```
# infra/terraform/backend.tf (create this file)
terraform {
  backend "s3" {
    bucket      = "networkpeer-terraform-state-prod"
    key         = "prod/terraform.tfstate"
    region     = "us-east-1"
    encrypt     = true
    dynamodb_table = "networkpeer-terraform-locks"
    profile     = "networkpeer-prod"
  }
}
```

## 3.3 Initialize

```
cd /Users/adityasharma/Desktop/NETWORKPEER/infra/terraform
terraform init -migrate-state
```

---

# 4. Infrastructure Deployment

---

## 4.1 Prepare Variables

```
cp terraform.tfvars.example terraform.tfvars
# Edit terraform.tfvars with production values:
# - db_password (use Secrets Manager ARN after creation)
# - cognito_callback_urls (your domain)
# - allowed_origins (your domain)
# - vpc_cidr, subnet_cidrs (if custom VPC)
```

## 4.2 Key Variables (terraform.tfvars)

```
# Required
aws_region      = "us-east-1"
environment     = "prod"
db_instance_class = "db.t3.medium"
db_allocated_storage = 100
redis_node_type  = "cache.t3.medium"
ecs_cpu          = 512
ecs_memory       = 1024
desired_count    = 2

# Cognito
cognito_domain_prefix = "networkpeer-prod"
callback_urls         = ["https://app.networkpeer.com/auth/callback"]
logout_urls           = ["https://app.networkpeer.com"]
allowed_origins       = ["https://app.networkpeer.com"]

# Domain (if using custom domain)
domain_name          = "networkpeer.com"
hosted_zone_id       = "Z123456789" # Route53 hosted zone ID

# Monitoring
alert_email          = "ops@networkpeer.com"
```

## 4.3 Plan & Apply

```
# Format & validate
terraform fmt -check -recursive
terraform validate

# Plan
terraform plan -out=tfplan

# Review plan carefully - should show:
# - VPC, subnets, NAT gateways
# - RDS PostgreSQL (Multi-AZ)
# - ElastiCache Redis
# - ECS Cluster + Service + Task Definition
# - ALB + Target Groups + Listener (HTTPS)
# - Cognito User Pool + Client + Custom Auth Lambdas
# - SNS Topic for SMS
# - S3 Buckets (evidence, terraform state)
# - IAM Roles (ECS Task, Lambda, GitHub Actions)
# - CloudWatch Log Groups, Alarms, Dashboards

# Apply
terraform apply tfplan
```

## 4.4 Expected Outputs

```
terraform output
# Save these for application config:
# - alb_dns_name
# - db_endpoint
# - redis_endpoint
# - cognito_user_pool_id
# - cognito_client_id
# - cognito_domain
# - evidence_bucket_name
# - ecs_task_execution_role_arn
# - ecs_task_role_arn
```

## 5. Database Provisioning & Migrations

### 5.1 Verify RDS Connectivity

```
# Get endpoint
DB_ENDPOINT=$(terraform output -raw db_endpoint)

# Test connection (from bastion or VPN)
psql "postgresql://postgres:${DB_PASSWORD}@${DB_ENDPOINT}:5432/postgres" -c
"SELECT version();"

```

### 5.2 Create Application Database & User

```
-- Run as postgres superuser
CREATE DATABASE networkpeer;
CREATE USER networkpeer_app WITH ENCRYPTED PASSWORD
'secure_password_from_secrets_manager';
GRANT ALL PRIVILEGES ON DATABASE networkpeer TO networkpeer_app;

-- Enable extensions
\c networkpeer
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS "uuid-ossf";
CREATE EXTENSION IF NOT EXISTS pg_trgm;

```

### 5.3 Store Credentials in Secrets Manager

```
aws secretsmanager create-secret \
  --name "prod/networkpeer/database" \
  --description "NetworkPeer Production Database Credentials" \
  --secret-string
    '{"username":"networkpeer_app","password":"secure_password","host":"'${DB_ENDPOINT}'"'
  \
  --profile networkpeer-prod

```

## 5.4 Run Migrations

```
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-main

# Set environment
export DATABASE_URL="postgresql://networkpeer_app:secure_password@${DB_ENDPOINT}:5432/networkpeer"
export NODE_ENV=production

# Run migrations in order
npm run migrate

# Verify all 40 migrations applied
psql "$DATABASE_URL" -c "SELECT * FROM schema_migrations ORDER BY version;"
```

## 5.5 Critical Migration: Cognito Identity Mapping

```
# Migration 040 creates the cognito_sub → user_id mapping table
# This is REQUIRED for Cognito Custom Auth to work
psql "$DATABASE_URL" -f migrations/040_add_cognito_identity_mapping.sql

# Verify
psql "$DATABASE_URL" -c "\d cognito_identity_mapping"
```

## 5.6 Seed Data (Optional - Demo)

```
# Run demo seed if needed
psql "$DATABASE_URL" -f scripts/seed-demo-data.sql
```

## 6. Cognito Custom Auth Configuration

---

### 6.1 Verify Terraform-Created Resources

```
# Get Cognito details
USER_POOL_ID=$(terraform output -raw cognito_user_pool_id)
CLIENT_ID=$(terraform output -raw cognito_client_id)
DOMAIN=$(terraform output -raw cognito_domain)

# Verify User Pool
aws cognito-idp describe-user-pool --user-pool-id $USER_POOL_ID --profile
    networkpeer-prod

# Verify Client (no secret)
aws cognito-idp describe-user-pool-client --user-pool-id $USER_POOL_ID --
    client-id $CLIENT_ID --profile networkpeer-prod
```

### 6.2 Verify Lambda Triggers

```
# List triggers
aws cognito-idp describe-user-pool --user-pool-id $USER_POOL_ID --profile
    networkpeer-prod \
    --query 'UserPool.LambdaConfig'

# Should show:
# - DefineAuthChallenge
# - CreateAuthChallenge
# - VerifyAuthChallengeResponse
# - PreTokenGeneration (for custom claims)
```

## 6.3 Test Custom Auth Flow (CLI)

```
# 1. Initiate Auth
aws cognito-idp initiate-auth \
  --auth-flow CUSTOM_AUTH \
  --client-id $CLIENT_ID \
  --auth-parameters USERNAME="+15551234567",ROLE="CLIENT" \
  --profile networkpeer-prod

# Response: ChallengeParameters {CHALLENGE_ID, ...}, Session

# 2. Respond to Challenge (use OTP from SMS)
aws cognito-idp respond-to-auth-challenge \
  --client-id $CLIENT_ID \
  --challenge-name CUSTOM_CHALLENGE \
  --session <session_from_step_1> \
  --challenge-responses
    USERNAME="+15551234567",ANSWER="123456",CHALLENGE_ID="<challenge_id>" \
  --profile networkpeer-prod

# Response: AuthenticationResult {AccessToken, IdToken, RefreshToken,
    ExpiresIn}
```

## 6.4 Configure Groups (CLIENT, WORKER, ADMIN)

```
# Groups created by Terraform, verify:
aws cognito-idp list-groups --user-pool-id $USER_POOL_ID --profile networkpeer-
  prod

# Assign user to group (after first login):
aws cognito-idp admin-add-user-to-group \
  --user-pool-id $USER_POOL_ID \
  --username "<cognito_sub>" \
  --group-name "CLIENT" \
  --profile networkpeer-prod
```

## 7. SNS SMS Setup

---

### 7.1 Verify SNS Configuration

```
# Check SMS attributes
aws sns get-sms-attributes --profile networkpeer-prod

# Required attributes:
# - DefaultSMSType: "Transactional" (for OTP)
# - MonthlySpendLimit: "$100" (or higher)
# - DeliveryStatusIAMRole: ARN for CloudWatch Logs
```

### 7.2 Configure Origination Identity (Required for Production)

```
# Option 1: Toll-free number (recommended for US)
aws sns create-origination-identity \
  --origination-identity "arn:aws:sns:us-east-1:123456789012:origination-identity/phone-number/+18005551234" \
  --profile networkpeer-prod

# Option 2: 10DLC (10-digit long code) - requires campaign registration
# Option 3: Short code - expensive, high volume
```

### 7.3 Test SMS Delivery

```
aws sns publish \
  --phone-number "+15551234567" \
  --message "Test OTP: 123456" \
  --message-attributes '{"AWS.SNS.SMS.SMSType": {"DataType": "String", "StringValue": "Transactional"}}' \
  --profile networkpeer-prod
```

## 8. Application Deployment

---

### 8.1 Build Docker Images

```
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-main

# Build API image
docker build -t networkpeer-api:prod .

# Tag for ECR
ECR_URI=$(aws ecr describe-repositories --repository-names networkpeer-api --
        profile networkpeer-prod --query 'repositories[0].repositoryUri' --
        output text)
docker tag networkpeer-api:prod $ECR_URI:prod

# Push
aws ecr get-login-password --profile networkpeer-prod | docker login --
        username AWS --password-stdin $ECR_URI
docker push $ECR_URI:prod
```

### 8.2 Update ECS Service

```
# Force new deployment
aws ecs update-service \
    --cluster networkpeer-prod \
    --service networkpeer-api \
    --force-new-deployment \
    --profile networkpeer-prod
```

## 8.3 Deploy Web Frontend

```
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-platform-main

# Build
npm run build

# Deploy to S3 + CloudFront (or Vercel/Netlify)
aws s3 sync dist/ s3://networkpeer-web-prod --delete --profile networkpeer-prod
aws cloudfront create-invalidation --distribution-id $DISTRIBUTION_ID --paths
    "/*" --profile networkpeer-prod
```

## 8.4 Configure Environment Variables (ECS Task Definition)

```
{
  "environment": [
    {"name": "NODE_ENV", "value": "production"},
    {"name": "DATABASE_URL", "valueFrom": "arn:aws:secretsmanager:us-east-1:123456789012:secret:prod/networkpeer/database"},
    {"name": "REDIS_URL", "value": "redis://networkpeer-prod.xxxxxx.use1.cache.amazonaws.com:6379"},
    {"name": "COGNITO_USER_POOL_ID", "value": "us-east-1_XXXXXXXX"},
    {"name": "COGNITO_CLIENT_ID", "value": "xxxxxxxxxxxxxxxxxxxxxxxxxxxx"},
    {"name": "COGNITO_REGION", "value": "us-east-1"},
    {"name": "EVIDENCE_BUCKET", "value": "networkpeer-evidence-prod"},
    {"name": "LOG_PRETTY", "value": "false"},
    {"name": "CORS_ORIGIN", "value": "https://app.networkpeer.com"}
  ],
  "secrets": [
    {"name": "JWT_SECRET", "valueFrom": "arn:aws:secretsmanager:...:jwt-secret"},
    {"name": "SENTRY_DSN", "valueFrom": "arn:aws:secretsmanager:...:sentry-dsn"}
  ]
}
```

## 9. Monitoring & Alerting

### 9.1 Key Metrics to Monitor

| Metric                    | Source      | Alert Threshold |
|---------------------------|-------------|-----------------|
| HTTPCode_Target_5XX_Count | ALB         | > 10/min        |
| TargetResponseTime p99    | ALB         | > 2s            |
| CPUUtilization            | ECS Service | > 80% 5min      |
| MemoryUtilization         | ECS Service | > 85% 5min      |
| DatabaseConnections       | RDS         | > 80% max       |
| FreeableMemory            | RDS         | < 500MB         |
| CurrConnections           | ElastiCache | > 80% max       |
| ThrottledRequests         | Lambda      | > 0             |
| SMSMonthToDateSpent       | SNS         | > \$80/month    |

### 9.2 Create Dashboards

```
# CloudWatch Dashboard JSON (import via console or CLI)
aws cloudwatch put-dashboard \
  --dashboard-name "NetworkPeer-Production" \
  --dashboard-body file://monitoring/dashboard.json \
  --profile networkpeer-prod
```

## 9.3 Log Insights Queries

```
-- Auth failures
fields @timestamp, @message
| filter @message like /AUTH_INVALID_OTP/
| stats count() by bin(5m)

-- Slow queries
fields @timestamp, @message
| filter @message like /slow query/
| stats avg(duration_ms) by query_type
```

---

## 10. Rollback Procedures

### 10.1 Application Rollback

```
# Get previous task definition
aws ecs describe-services --cluster networkpeer-prod --services networkpeer-
api --profile networkpeer-prod

# Update to previous task definition ARN
aws ecs update-service \
  --cluster networkpeer-prod \
  --service networkpeer-api \
  --task-definition networkpeer-api:42 \
  --profile networkpeer-prod
```

## 10.2 Database Rollback

```
# Restore from snapshot (RDS)
aws rds restore-db-instance-from-db-snapshot \
  --db-instance-identifier networkpeer-prod-restored \
  --db-snapshot-identifier networkpeer-prod-snapshot-20260901 \
  --profile networkpeer-prod

# Update ALB target group to new DB endpoint (via Terraform)
terraform apply -var="db_endpoint=new-endpoint"
```

## 10.3 Terraform Rollback

```
# Revert to previous state version
terraform state pull > backup.tfstate
# Manually edit or use versioned state in S3
terraform apply -state=backup.tfstate
```

---

# 11. Production Checklist (Pre-Go-Live)

---

### Infrastructure

- ☐ Terraform apply successful, no drift
- ☐ RDS Multi-AZ enabled, backup retention 30 days
- ☐ ElastiCache Multi-AZ enabled
- ☐ ALB HTTPS listener with valid ACM certificate
- ☐ Security groups least-privilege (ALB→ECS, ECS→RDS/Redis)
- ☐ VPC Flow Logs enabled
- ☐

GuardDuty enabled

## Cognito

- ☐ User Pool MFA optional (can enable later)
- ☐ Custom Auth Lambdas deployed and tested
- ☐ SNS SMS production access approved
- ☐ Origination identity configured
- ☐ Callback URLs match production domain
- ☐ Groups: CLIENT, WORKER, ADMIN exist

## Application

- ☐ Docker images built and pushed to ECR
- ☐ ECS service running desired count
- ☐ Health checks passing (ALB target group healthy)
- ☐ Environment variables set (no plaintext secrets)
- ☐ Database migrations applied (including #040)
- ☐ Web frontend deployed and accessible

## Security

- ☐ No hardcoded secrets in images/config
- ☐ TLS 1.2+ enforced everywhere
- ☐

CORS restricted to production domain

☐

Rate limits configured and tested

☐

WAF rules attached to ALB (optional but recommended)

## Monitoring

☐

CloudWatch alarms created and tested

☐

Log groups retention set

☐

SNS topic for alerts subscribed

☐

Dashboard operational

## Mobile Apps

☐

iOS: Bundle ID matches Cognito client

☐

Android: Package name matches Cognito client

☐

Both apps point to production API endpoint

☐

Push notification certificates/keys configured

## 12. Cost Estimation (Monthly, us-east-1)

| Service        | Configuration               | Est. Cost           |
|----------------|-----------------------------|---------------------|
| ECS Fargate    | 2 tasks × 0.5 vCPU / 1GB    | ~\$45               |
| ALB            | 1 LCU                       | ~\$25               |
| RDS PostgreSQL | db.t3.medium Multi-AZ 100GB | ~\$180              |
| ElastiCache    | cache.t3.medium Multi-AZ    | ~\$70               |
| Cognito        | 10K MAU                     | ~\$0 (free tier)    |
| Lambda         | 1M invocations              | ~\$5                |
| SNS SMS        | 10K messages                | ~\$7.50             |
| S3             | 100GB storage + requests    | ~\$5                |
| CloudWatch     | Logs + Metrics + Alarms     | ~\$30               |
| Data Transfer  | ~50GB                       | ~\$4.50             |
| <b>Total</b>   |                             | <b>~\$372/month</b> |

## 13. Emergency Contacts

| Role          | Name       | Contact           | Escalation          |
|---------------|------------|-------------------|---------------------|
| Platform Lead |            |                   | Primary             |
| Backend Lead  |            |                   | Secondary           |
| DevOps Lead   |            |                   | Infrastructure      |
| Security Lead |            |                   | Security incidents  |
| AWS Support   | Enterprise | Console → Support | Critical production |

*This runbook is a living document. Update after each deployment.*